HERANÇA

Prof. André Backes

Definição

- Conceito fundamental da programação orientada a objetos
- Permite a criação de novas classes a partir de classes existentes
  - Reaproveita e estende o comportamento da classe original

Definição

- A classe derivada herda os atributos e métodos da classe base, podendo usá-los diretamente ou adaptá-los às suas necessidades
  - Evita repetição de código
  - Permite que conceitos comuns fiquem concentrados em um único lugar

Relação hierárquica

- É uma relação hierárquica entre classes, na qual uma classe mais genérica serve como base para classes mais especializadas
- Cria relações do tipo “é um” entre as classes
  - No topo da hierarquia: classes mais abstratas e genéricas
  - Nas camadas inferiores: classes mais concretas e especializadas

Relação hierárquica

- Um cão “é um” animal
  - Porém, possui características específicas que o diferenciam de outros animais
- “Cão” é uma subclasse de “Animal”
  - “Cão” herdará as características da classe “Animal”, além de poder adicionar novos membros específicos de cães, como “raça”
- Essa organização facilita a compreensão do sistema
  - Comportamentos comuns ficam em um ponto central da hierarquia
  - Facilita a reutilização de código

Herança | Cuidados

- Embora seja um recurso poderoso, ela deve ser usada com cuidado
  - Uma hierarquia mal planejada pode gerar acoplamento excessivo entre classes e dificultar alterações futuras
    - Quanto mais alto o grau de acoplamento, mais dependente uma classe é da outra
  - Isso pode tornar o sistema mais difícil de entender e dar manutenção, pois alterações em uma classe podem afetar outras classes do sistema

Herança | Cuidados

- *“Se um gato possui **raça** e **patas**, e um cachorro possui **raça**, **patas** e **tipoDoPelo**, logo **Cachorro extends Gato**? Pode parecer engraçado, mas é o mesmo caso que os anteriores: herança por preguiça, por comodismo, por que vai dar uma ajudinha. A relação “é um” não se encaixa aqui, e vai nos gerar problemas.”*
  - *https://www.alura.com.br/artigos/como-nao-aprender-orientacao-a-objetos-heranca*
  - https://objectpascalprogramming.com/posts/heranca-pode-ser-o-mal-da-orientacao-a-objetos-parte-1/

Herança | Cuidados

- Se mal feita, pode gerar problemas
  - Existe um aviso grande no *javadoc* para ninguém usar determinados métodos herdados

Because **`Properties`** inherits from **`Hashtable`**, the **`put`** and **`putAll`** methods can be applied to a **`Properties`** object. Their use is strongly discouraged as they allow the caller to insert entries whose keys or values are not **`Strings`**. The **`setProperty`** method should be used instead. If the **`store`** or **`save`** method is called on a "compromised" **`Properties`** object that contains a non-**`String`** key or value, the call will fail. Similarly, the call to the **`propertyNames`** or **`list`** method will fail if it is called on a "compromised" **`Properties`** object that contains a non-**`String`** key.

Herança x Relacionamentos

- A principal diferença está no grau de acoplamento
  - Herança
    - Vínculo mais rígido entre classes, pois a derivada depende diretamente da estrutura da base
  - Composição e agregação
    - Vínculos mais flexíveis, permitindo que as classes mudem independentemente, desde que mantenham a interface de comunicação

Herança x Relacionamentos

- Herança, relação “é um”
  - Relação estrutural e hierárquica
    - A classe derivada herda características da classe base e pode especializá-las
    - Carro herda as características de veículo, mas acrescenta detalhes como porta-malas e marchas
- Composição e agregação, relação “tem um”
  - Relação onde um pode existir sem o outro ou é parte
    - Um Carro tem um Motor: o motor é parte fundamental do carro
    - Uma Universidade tem Professores: os professores estão associados, mas podem existir independentemente da universidade

Criando uma herança entre classes

- Classe Pessoa
  - Atributos privados nome e idade
- Classe Jogador
  - Atributos modalidade esportiva e equipe
  - Representação incompleta
    - Não possui nome e idade
    - Pessoa possui esses atributos!

```
class Pessoa {
private:
    string nome;
    int idade;
public:
    Pessoa(string nome, int idade)
        : nome(nome), idade(idade) {}

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
    }
};

class Jogador {
private:
    string esporte;
    string equipe;
public:
    Jogador(string esporte, string equipe)
        : esporte(esporte), equipe(equipe) {}

    void exibirDados() {
        cout << "Esporte: " << esporte << "\nEquipe: " << equipe << endl;
    }
};
```

Criando uma herança entre classes

- Incluir um objeto Pessoa dentro de Jogador?
  - É possível, mas não faz muito sentido
    - Jogador não “tem uma” pessoa, Jogador “é uma” Pessoa
  - O correto é que Jogador herde as propriedades e características de Pessoa na sua implementação

```
class Pessoa {
private:
    string nome;
    int idade;
public:
    Pessoa(string nome, int idade)
        : nome(nome), idade(idade) {}

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
    }
};

class Jogador {
private:
    string esporte;
    string equipe;
public:
    Jogador(string esporte, string equipe)
        : esporte(esporte), equipe(equipe) {}

    void exibirDados() {
        cout << "Esporte: " << esporte << "\nEquipe: " << equipe << endl;
    }
};
```

Criando uma herança entre classes

- Vamos incluir Pessoa na declaração de Jogador
  - Herança pública (**public**)
  - Atributos de Pessoa agora são **protected**
    - Atributos e métodos são privados, mas ainda podem ser acessados por classes derivadas
    - Jogador pode acessar os atributos nome e idade na sua função exibirDados()

```
class Pessoa {
protected:
    string nome;
    int idade;
public:
    Pessoa(string nome = "Anonimo", int idade = 0)
        : nome(nome), idade(idade) {}

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
    }
};

class Jogador: public Pessoa {
private:
    string esporte;
    string equipe;
public:
    Jogador(string esporte, string equipe)
        : esporte(esporte), equipe(equipe) {}

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
        cout << "Esporte: " << esporte << "\nEquipe: " << equipe << endl;
    }
};
```

Criando uma herança entre classes

- Mudança no construtor de Pessoa
  - Definição de valores padrão para seus atributos
    - Evita erro de compilação
  - Jogador “ainda” não sabe como inicializar os atributos da classe base

```
class Pessoa {
protected:
    string nome;
    int idade;
public:
    Pessoa(string nome = "Anonimo", int idade = 0)
        : nome(nome), idade(idade) {}

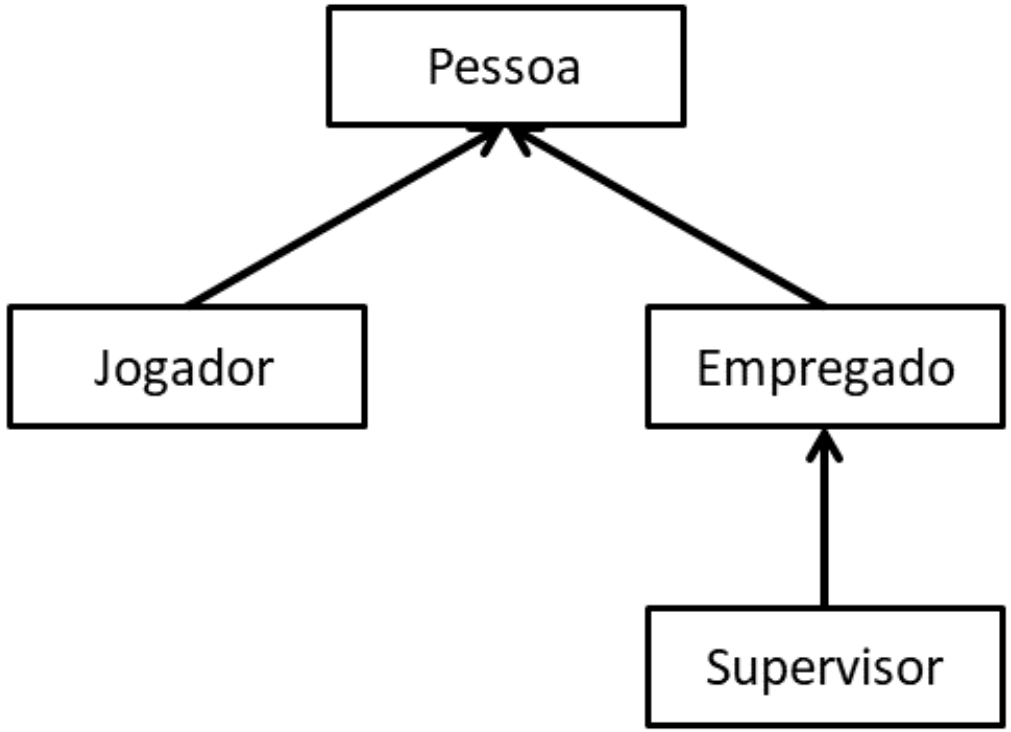
    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
    }
};

class Jogador: public Pessoa {
private:
    string esporte;
    string equipe;
public:
    Jogador(string esporte, string equipe)
        : esporte(esporte), equipe(equipe) {}

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
        cout << "Esporte: " << esporte << "\nEquipe: " << equipe << endl;
    }
};
```

Representação hierárquica

- Uma classe base pode dar origem a várias classes derivadas
  - Empregado e Jogador herdam de Pessoa
- Uma classe derivada pode ser a base para outra classe derivada
  - Supervisor herda de Empregado



Criando uma herança entre classes

- Podemos criar uma classe derivada a partir de uma outra classe derivada
  - Supervisor “é um” tipo especial de Empregado
  - Empregado já é uma derivação da classe Pessoa

```
class Empregado : public Pessoa {
private:
    double salario;
    string cargo;
public:
    Empregado(double salario, string cargo)
        : salario(salario), cargo(cargo) {}

    void exibirDados() {
        cout << "Salário: " << salario << "\nCargo: " << cargo << endl;
    }
};

class Supervisor : public Empregado {
private:
    int idsSupervisionados[5]{};
};
```

ACESSO E SUPERPOSIÇÃO DE MÉTODOS

Adicionando funcionalidades a classe derivada

- A herança permite
  - Reutilizar o código da classe base
  - Implementar funcionalidades na classe derivada
- Classe Circulo
  - Herda os atributos x e y de Ponto
  - Define novos atributos e funcionalidades
    - raio
    - setRaio()
    - imprime()

```
class Ponto{
protected:
    int x, y;
public:
    Ponto(int _x=0, int _y=0): x{_x},y{_y}{}
    void exibir() {
        cout << "Ponto\n";
    }
};

class Circulo : public Ponto{
protected:
    int raio;
public:
    Circulo(int _x, int _y, int _r):
        Ponto(_x,_y), raio{_r}{}

    void setRaio(int _r){raio = _r;}

    void imprime(){
        cout << "(" << x << ", ";
        cout << y << ", " << raio << ")\n";
    }
};
```

Acesso e superposição de métodos

- A herança favorece o reaproveitamento de código
  - Métodos públicos da classe base permanecem acessíveis
    - Podem ser chamados pela classe derivada sem necessidade de reimplementação
    - Método **exibir()**
  - Métodos **protected** são acessíveis apenas dentro da classe derivada

```
class Ponto{
protected:
    int x, y;
public:
    Ponto(int _x=0, int _y=0): x{_x},y{_y}{}
    void exibir() {
        cout << "Ponto\n";
    }
};

class Circulo : public Ponto{
protected:
    int raio;
public:
    Circulo(int _x, int _y, int _r):
        Ponto(_x,_y), raio{_r}{}
};

int main() {
    Ponto p(0,0);
    p.exibir();

    Circulo c(0,0,1);
    c.exibir();

    return 0;
}
```

Acesso e superposição de métodos

- Superposição de métodos
  - A classe derivada define um método com o mesmo nome de um método presente na classe base
    - Número de parâmetros e tipo de retorno podem ser diferentes
    - O método da base pode ser ou não virtual
  - Muda a implementação de uma função da classe base na classe derivada

```
class Base {
public:
    void mostrar() {
        cout << "Base\n";
    }

    void mostrar(int n) {
        cout << "Base: " << n << "\n";
    }
};

class Derivada : public Base {
public:
    void mostrar() {
        cout << "Derivada\n";
    }
};

int main() {
    Base b;
    b.mostrar();

    Derivada d;
    d.mostrar();

    //Erro de compilação
    //d.mostrar(5);

    return 0;
}
```

Acesso e superposição de métodos

- Superposição de métodos
  - Método herdado da base deixa de ser diretamente visível
    - O compilador aplica as regras de resolução de nomes
    - Quando encontra no escopo da derivada um método com determinado nome, ele ignora as versões herdadas da base com o mesmo nome
  - Oculta todas as versões do método na classe Base
    - Erro de compilação se a classe Derivada tentar chamar **mostrar(5)**

```
class Base {
public:
    void mostrar() {
        cout << "Base\n";
    }

    void mostrar(int n) {
        cout << "Base: " << n << "\n";
    }
};

class Derivada : public Base {
public:
    void mostrar() {
        cout << "Derivada\n";
    }
};

int main() {
    Base b;
    b.mostrar();

    Derivada d;
    d.mostrar();

    //Erro de compilação
    //d.mostrar(5);

    return 0;
}
```

Acesso e superposição de métodos

- Diretiva **using**
  - Permite acessar as funções ocultas da base
  - Permite que Derivada continue usando a sua implementação do método **mostrar()**, mas deixa visível as outras sobrecargas existentes para o método
    - Resolve o problema de tentar chamar **mostrar(5)**

```
class Base {
public:
    void mostrar() {
        cout << "Base\n";
    }

    void mostrar(int n) {
        cout << "Base: " << n << "\n";
    }
};

class Derivada : public Base {
public:
    // reexpõe todas as versões de 'mostrar' da base
    using Base::mostrar;
    void mostrar() {
        cout << "Derivada\n";
    }
};

int main() {
    Base b;
    b.mostrar();

    Derivada d;
    d.mostrar();
    d.mostrar(5);

    return 0;
}
```

Acesso e superposição de métodos

- Operador de resolução de escopo
  - Permite chamar a implementação original mesmo com superposição
  - Base::mostrar()
    - Força a chamada ao método da classe base
    - Ignora o fato de que ele foi escondido pela derivada

```
class Base {
public:
    void mostrar() {
        cout << "Base\n";
    }

    void mostrar(int n) {
        cout << "Base: " << n << "\n";
    }
};

class Derivada : public Base {
public:
    void mostrar() {
        cout << "Derivada\n";
    }
};

int main() {
    Derivada d;
    d.mostrar();
    d.Base::mostrar();
    d.Base::mostrar(5);

    return 0;
}
```

Ocultando um método na classe derivada

- Diretiva **using** possui um uso mais amplo
  - Permite trazer um método da classe base para o escopo da classe derivada com uma nova visibilidade
  - método **mostrar()**
    - Público na classe base
    - Privado na classe derivada

```
class Base {
public:
    void mostrar() {
        cout << "Base\n";
    }
};

class Derivada : public Base {
private:
    using Base::mostrar;
};

int main() {
    Base b;
    b.mostrar();

    Derivada d;
    //Erro de compilação
    d.mostrar();

    return 0;
}
```

Ocultando um método na classe derivada

- Operador **delete** (padrão C++11)
  - Indica ao compilador que o método foi explicitamente removido e não pode ser chamado pela classe derivada
  - Definido na assinatura do método

```
class Base {
public:
    void mostrar() {
        cout << "Base\n";
    }
};

class Derivada : public Base {
public:
    void mostrar() = delete;
};

int main() {
    Base b;
    b.mostrar();

    Derivada d;
    //Erro de compilação
    d.mostrar();

    return 0;
}
```

CONSTRUTORES E DESTRUTORES

Construção das classes derivadas

- Para construir um objeto da classe derivada é necessário percorrer toda a hierarquia da herança
  - Primeiro é chamado o construtor da classe base
    - Ou das classes base, caso haja mais de uma
  - Depois são construídos os atributos membros da classe derivada
    - Se existirem
  - Por último é chamado o construtor da própria classe derivada

Construção das classes derivadas

- O processo de construção é feito de cima para baixo na hierarquia
  - Sempre do mais genérico para o mais específico
- Assim, quando o construtor da classe derivada for executado
  - Toda a base já esteja pronta
  - Todos os atributos herdados estejam inicializados

Construção das classes derivadas

- Duas classes simples: Base e Derivada
  - Mensagem simples no construtor
  - Nenhum atributo
- Criação de um objeto (saída)
  - Classe Base
    - `Construtor da Base`
  - Classe Derivada
    - `Construtor da Base`
    - `Construtor da Derivada`

```
class Base {
public:
    Base() {
        cout << "Construtor da Base\n";
    }
};

class Derivada : public Base {
public:
    Derivada() {
        cout << "Construtor da Derivada\n";
    }
};

int main() {
    Base ba;
    Derivada de;
    return 0;
}
```

Construção das classes derivadas

- Se tivermos mais níveis, a ordem de chamada seguirá a hierarquia inteira
- Criação de um objeto (saída)
  - Classe Pessoa
    - `Construtor de Pessoa`
  - Classe Empregado
    - `Construtor de Pessoa`
    - `Construtor de Empregado`
  - Classe Supervisor
    - `Construtor de Pessoa`
    - `Construtor de Empregado`
    - `Construtor de Supervisor`

```
class Pessoa {
protected:
    string nome;
    int idade;
public:
    Pessoa(string nome = "Anonimo", int idade = 0)
        : nome(nome), idade(idade) {
        cout << "Construtor de Pessoa \n";
    }
};

class Empregado : public Pessoa {
private:
    double salario;
    string cargo;
public:
    Empregado(double salario = 0, string cargo = "NULL")
        : salario(salario), cargo(cargo) {
        cout << "Construtor de Empregado \n";
    }
};

class Supervisor : public Empregado {
private:
    int idsSupervisionados[5]{};
public:
    Supervisor(){
        cout << "Construtor de Supervisor \n";
    }
};
```

Inicialização das classes derivadas

- Construtor de Pessoa
  - Definição de valores padrão para seus atributos
    - Evita erro de compilação
  - Empregado “ainda” não sabe como inicializar os atributos da classe base

```
class Pessoa {
protected:
    string nome;
    int idade;
public:
    Pessoa(string nome = "Anonimo", int idade = 0)
        : nome(nome), idade(idade) {}

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
    }
};

class Empregado : public Pessoa {
private:
    double salario;
    string cargo;
public:
    Empregado(double salario, string cargo)
        : salario(salario), cargo(cargo) {}

    void exibirDados() {
        cout << "Salário: " << salario << "\nCargo: " << cargo << endl;
    }
};

int main() {
    Pessoa pessoa("Carlos", 25);
    Empregado emp(3500,"Caixa");

    return 0;
}
```

Inicialização das classes derivadas

- Por que Empregado não sabe como inicializar os atributos da classe base?
  - A responsabilidade de inicializar os atributos da classe base é sempre do construtor da própria classe base
  - Conceitualmente, a classe derivada não deve conhecer detalhes internos da implementação da base

Inicialização das classes derivadas

- Porém, quando criamos um objeto da classe derivada, podemos querer definir os valores para os atributos da classe base
  - A inicialização de atributos da classe base pode ser feita de três maneiras
    - Construtor com valores pré-definidos
    - Construtor parametrizado com inicialização no corpo do construtor;
    - Construtor parametrizado e delegar a inicialização dos atributos da classe base

Inicialização das classes derivadas

- Usar o corpo do construtor para atribuir valores tem limitações
  - Os atributos base devem ser **protected**
  - A base não pode ter um construtor
    - O que não é ideal

```
class Pessoa {
protected:
    string nome;
    int idade;
public:
    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
    }
};

class Empregado : public Pessoa {
private:
    double salario;
    string cargo;
public:
    Empregado(string _nome, int _idade,
              double _salario, string _cargo)
        : salario(_salario), cargo(_cargo) {
            nome = _nome;
            idade = _idade;
        }

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
        cout << "Salário: " << salario << "\nCargo: " << cargo << endl;
    }
};

int main() {
    Empregado emp("Carlos", 25, 3500, "Caixa");
    emp.exibirDados();

    return 0;
}
```

Inicialização das classes derivadas

- Usar o corpo do construtor para atribuir valores tem limitações
  - Se a base tiver construtor
    - Precisa de valores pré-definidos
    - Atributos são inicializados duas vezes
      - Com os valores pré-definidos (instanciação)
      - Com os valores do corpo do construtor
      - Isso é algo indesejável

```
class Pessoa {
protected:
    string nome;
    int idade;
public:
    Pessoa(string _nome = "Anonimo", int _idade = 0)
        : nome(_nome), idade(_idade) {}

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
    }
};

class Empregado : public Pessoa {
private:
    double salario;
    string cargo;
public:
    Empregado(string _nome, int _idade,
              double _salario, string _cargo)
        : salario(_salario), cargo(_cargo) {
            nome = _nome;
            idade = _idade;
        }

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
        cout << "Salário: " << salario << "\nCargo: " << cargo << endl;
    }
};
```

Inicialização das classes derivadas

- Podemos delegar ao construtor da classe base a inicialização dos seus atributos
  - Configura o construtor da derivada para chamar o construtor da base
    - Reutiliza a lógica de inicialização
    - Os atributos da base não podem ser acessados na lista de inicialização do construtor da derivada

```
class Pessoa {
protected:
    string nome;
    int idade;
public:
    Pessoa(string _nome = "Anonimo", int _idade = 0)
        : nome(_nome), idade(_idade) {}

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
    }
};

class Empregado : public Pessoa {
private:
    double salario;
    string cargo;
public:
    Empregado(string _nome, int _idade,
              double _salario, string _cargo)
        : Pessoa(_nome, _idade), salario(_salario), cargo(_cargo) { }

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
        cout << "Salário: " << salario << "\nCargo: " << cargo << endl;
    }
};
```

Inicialização das classes derivadas

- Delegar o construtor é a melhor solução
  - Permite escolher o construtor da classe base
    - Podem existir vários
  - Mantém o encapsulamento
    - Podemos inicializar atributos privados
  - Evita que a derivada conheça detalhes internos da base

```
class Pessoa {
protected:
    string nome;
    int idade;
public:
    Pessoa(string _nome = "Anonimo", int _idade = 0)
        : nome(_nome), idade(_idade) {}

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
    }
};

class Empregado : public Pessoa {
private:
    double salario;
    string cargo;
public:
    Empregado(string _nome, int _idade,
              double _salario, string _cargo)
        : Pessoa(_nome, _idade), salario(_salario), cargo(_cargo) { }

    void exibirDados() {
        cout << "Nome: " << nome << "\nIdade: " << idade << endl;
        cout << "Salário: " << salario << "\nCargo: " << cargo << endl;
    }
};
```

Destruindo a classe derivada

- O processo acontece na ordem inversa da construção
  - Primeiro é chamado o destrutor da classe derivada
  - Depois são destruídos os atributos membros da derivada
    - se houver
  - Por último é chamado o destrutor da classe base

Destruindo a classe derivada

- O processo é feito de baixo para cima na hierarquia
  - Sempre do mais específico para o mais genérico
  - Garante a correta liberação de todos os recursos alocados e atributos herdados
    - Se a classe base fosse destruída antes da derivada, a derivada poderia tentar acessar atributos herdados que já não existiriam, causando comportamento indefinido

Destruindo a classe derivada

- Duas classes simples: Base e Derivada
  - Mensagem simples no construtor/destrutor
  - Nenhum atributo
- Criação/Destruição de um objeto (saída)
  - Classe Base
    - `Construtor da Base`
    - `Destrutor da Base`
  - Classe Derivada
    - `Construtor da Base`
    - `Construtor da Derivada`
    - `Destrutor da Derivada`
    - `Destrutor da Base`

```
class Base {
public:
    Base() {
        cout << "Construtor da Base\n";
    }
    ~Base() {
        cout << "Destrutor da Base\n";
    }
};

class Derivada : public Base {
public:
    Derivada() {
        cout << "Construtor da Derivada\n";
    }
    ~Derivada() {
        cout << "Destrutor da Derivada\n";
    }
};

int main() {
    Derivada de;
    return 0;
}
```

Destruindo a classe derivada

- Se tivermos mais níveis, a ordem de chamada seguirá a hierarquia inteira

```
class Pessoa {
protected:
    string nome;
    int idade;
public:
    Pessoa(string nome = "Anonimo", int idade = 0)
        : nome(nome), idade(idade) {
        cout << "Construtor de Pessoa \n";
    }
    ~Pessoa(){
        cout << "Destrutor de Pessoa \n";
    }
};

class Empregado : public Pessoa {
private:
    double salario;
    string cargo;
public:
    Empregado(double salario = 0, string cargo = "NULL")
        : salario(salario), cargo(cargo) {
        cout << "Construtor de Empregado \n";
    }
    ~Empregado(){
        cout << "Destrutor de Empregado \n";
    }
};
```

Destruindo a classe derivada

- Criação/destruição do objeto (saída)
  - Classe Supervisor
    - Construtor de Pessoa
    - Construtor de Empregado
    - Construtor de Supervisor
    - Destrutor de Supervisor
    - Destrutor de Empregado
    - Destrutor de Pessoa

```
class Supervisor : public Empregado {
private:
    int idsSupervisionados[5]{};
public:
    Supervisor(){
        cout << "Construtor de Supervisor \n";
    }
    ~Supervisor(){
        cout << "Destrutor de Supervisor \n";
    }
};

int main() {
    Supervisor sup;

    return 0;
}
```

TIPOS DE HERANÇA

Tipos de Herança

- Efeito direto nos modificadores de acesso dos atributos e funções da classe base
  - Permite alterar o modificador de acesso para um membro herdado (apenas na classe derivada) dependendo do tipo de herança utilizado
  - Isso afeta como esse atributo pode ser acessado em uma nova herança
  - Deve refletir a intenção de *design*

Tipos de Herança

- Três níveis de acesso possíveis
  - **public**
  - **protected**
  - **private**
- Até aqui vimos a herança pública (**public**)
  - É a mais comum para modelagem conceitual
    - Recomendada
  - A protegida e a privada são mais usadas para reuso de código sem expor detalhes

```
// Herança Pública
class DerivadaPublica: public Base
{
};

// Herança Protegina
class DerivadaProtegida: protected Base
{
};

// Herança Privada
class DerivadaPrivada: private Base
{
};
```

Tipos de Herança | Pública

- Mais comum e a mais recomendada
  - Relação clara do tipo “é um”
    - Carro “é um” Veiculo
  - Preserva a visibilidade original da classe base, facilitando reutilização
    - Membros públicos continuam públicos na derivada
    - Membros protegidos (**protected**) continuam protegidos na derivada
    - Membros privados (**private**) não são acessíveis diretamente na derivada

```
class Base {
public:
    void metodoPublico() {
        cout << "Método público da Base\n";
    }

protected:
    void metodoProtegido() {
        cout << "Método protegido da Base\n";
    }

private:
    void metodoPrivado() {
        cout << "Método privado da Base\n";
    }
};

class Derivada : public Base {
public:
    void teste() {
        // Acessível (continua público)
        metodoPublico();
        // Acessível (continua protegido)
        metodoProtegido();
        // Erro: inacessível
        // metodoPrivado();
    }
};
```

Tipos de Herança | Pública

- Na herança pública
  - A classe pode acessar os métodos públicos e protegidos da Base dentro dos seus métodos internos, **teste()**
  - Seu objeto pode chamar os métodos públicos da Base no programa principal
  - Apenas os métodos privados estão inacessíveis

```
int main() {
    Derivada pub;

    pub.metodoPublico();
    pub.teste();

    // Erro: protegido
    // pub.metodoProtegido();

    // Erro: inacessível
    // pub.metodoPrivado();

    return 0;
}
```

Tipos de Herança | Protegida

- Oculta a interface pública da base de usuários externos
  - A Base serve como implementação interna para a derivada
  - Fica disponível apenas para subclasses
    - Membros públicos passam a ser protegidos (**protected**) na derivada
    - Membros protegidos (**protected**) continuam protegidos na derivada
    - Membros privados (**private**) não são acessíveis diretamente na derivada

```
class Base {
public:
    void metodoPublico() {
        cout << "Método público da Base\n";
    }

protected:
    void metodoProtegido() {
        cout << "Método protegido da Base\n";
    }

private:
    void metodoPrivado() {
        cout << "Método privado da Base\n";
    }
};

class Derivada : protected Base {
public:
    void teste() {
        // Acessível internamente (vira protegido)
        metodoPublico();
        // Acessível internamente (continua protegido)
        metodoProtegido();
        // Erro: inacessível
        // metodoPrivado();
    }
};
```

Tipos de Herança | Protegida

- Na herança protegida
  - A classe pode acessar os métodos públicos e protegidos da Base dentro dos seus métodos internos, **teste()**
  - Um objeto da classe Derivada não possui acesso aos métodos da classe Base no programa principal
  - Métodos privados estão inacessíveis

```
int main() {
    Derivada prot;
    prot.teste();

    // Erro: virou protegido
    // prot.metodoPublico();

    // Erro: protegido
    // prot.metodoProtegido();

    // Erro: inacessível
    // prot.metodoPrivado();

    return 0;
}
```

Tipos de Herança | Privada

- Não expõe a interface da classe base
  - Para o mundo exterior, não existe relação “é um”, mas “é implementada usando”
    - Membros públicos e protegidos da base passam a ser privados na derivada
    - Os membros privados continuam inacessíveis diretamente

```
class Base {
public:
    void metodoPublico() {
        cout << "Método público da Base\n";
    }

protected:
    void metodoProtegido() {
        cout << "Método protegido da Base\n";
    }

private:
    void metodoPrivado() {
        cout << "Método privado da Base\n";
    }
};

class Derivada : private Base {
public:
    void teste() {
        // Acessível internamente (vira privado)
        metodoPublico();
        // Acessível internamente (vira privado)
        metodoProtegido();
        // Erro: inacessível
        // metodoPrivado();
    }
};
```

Tipos de Herança | Privada

- Na herança privada
  - A classe pode acessar os métodos públicos e protegidos da Base dentro dos seus métodos internos, **teste()**
  - Um objeto da classe Derivada não possui acesso aos métodos da classe Base no programa principal

```
int main() {
    Derivada priv;
    priv.teste();

    // Erro: virou privado
    // priv.metodoPublico();

    // Erro: virou privado
    // priv.metodoProtegido();

    // Erro: inacessível
    // priv.metodoPrivado();

    return 0;
}
```